

DATABASE MANAGEMENT SYSTEMS

HMC (Hostel Management System) - Retrieval Queries

INSTRUCTOR: PROF. PM JAT

GROUP MEMBERS:

1. ZALAK THAKKAR 202401482
2. DEVANSHI MODI 202401498
3. DARSHAN TITA 202401485
4. SHIVAM THANKI 202401483
5. QUINCY VADI 202401488

QUERY 1 : List all active students with their assigned room number and wing name.

```
SELECT s.student_id, s.name AS student_name, s.branch, s.year,  
r.room_no, w.wing_name
```

```
FROM Student s
```

```
JOIN Rooms r ON s.resident_of = r.room_no
```

```
JOIN Wing w ON r.wing_id = w.wing_id
```

```
WHERE s.status = 'Active'
```

```
ORDER BY w.wing_name, r.room_no;
```

QUERY 2 : Find all rooms that are currently available (not full) along with the number of students currently residing in each room.

```
SELECT r.room_no, w.wing_name, r.capacity, COUNT(s.student_id) AS
current_occupants, r.capacity - COUNT(s.student_id) AS vacant_seats
FROM Rooms r

JOIN Wing w ON r.wing_id = w.wing_id

LEFT JOIN Student s ON s.resident_of = r.room_no AND s.status =
'Active'

GROUP BY r.room_no, w.wing_name, r.capacity, r.availability

HAVING r.capacity - COUNT(s.student_id) > 0

ORDER BY vacant_seats DESC;
```

QUERY 3 : List students who have health issues/special conditions recorded.

```
SELECT s.student_id, s.name, s.contact, s.health_details, r.room_no, w.wing_name

FROM Student s

JOIN Rooms r ON s.resident_of = r.room_no

JOIN Wing w ON r.wing_id = w.wing_id

WHERE s.health_details IS NOT NULL
```

AND s.status = 'Active';

QUERY 4 : Count the number of active students per branch and year of study.

SELECT branch, year, COUNT(*) AS student_count

FROM Student

WHERE status = 'Active'

GROUP BY branch, year

ORDER BY branch, year;

QUERY 5 : Find students who are HMC committee members.

SELECT s.student_id, s.name, s.email, s.branch, s.year, w.wing_name

FROM Student s

JOIN Rooms r ON s.resident_of = r.room_no

JOIN Wing w ON r.wing_id = w.wing_id

WHERE s.hmc_flag = TRUE

AND s.status = 'Active'

ORDER BY w.wing_name;

QUERY 6: List all wings with total room count and total student capacity.

```
SELECT w.wing_id, w.wing_name, w.type, w.floors, COUNT(r.room_no) AS
total_rooms,

SUM(r.capacity) AS total_capacity

FROM Wing w

LEFT JOIN Rooms r ON r.wing_id = w.wing_id

GROUP BY w.wing_id, w.wing_name, w.type, w.floors

ORDER BY w.wing_name;
```

QUERY 7: List all staff members with their role and the name of their manager.

```
SELECT e.staff_id, e.name AS employee, e.role, e.salary, a.name AS
managed_by

FROM Staff e

JOIN Admin a ON e.managed_by = a.Admin_id

ORDER BY e.role, e.name;
```

QUERY 8 : List all wardens and the wing they are assigned to.

```
SELECT s.staff_id, s.name AS warden_name, s.contact_no,  
w.wing_name, w.type AS wing_type
```

```
FROM Warden wr
```

```
JOIN Staff s ON wr.staff_id = s.staff_id
```

```
JOIN Wing w ON wr.assigned_wing = w.wing_id;
```

QUERY 9 : List all cleaning staff and their working areas.

```
SELECT s.staff_id, s.name AS staff_name, s.contact_no, cs.working_area,  
s.working_hours, s.working_days
```

```
FROM Cleaning_Staff cs
```

```
JOIN Staff s ON cs.staff_id = s.staff_id;
```

QUERY 10 : Find the total salary expenditure by staff role.

```
SELECT role, COUNT(*) AS head_count, SUM(salary) AS total_salary, AVG(salary)  
AS avg_salary
```

```
FROM Staff
```

```
GROUP BY role
```

ORDER BY total_salary DESC;

QUERY 11 : Find the total salary expenditure by staff role.

SELECT s.staff_id, s.name, s.role

FROM Staff s

WHERE s.staff_id NOT IN (

 SELECT solved_by FROM Complaint WHERE solved_by IS NOT NULL

 UNION

 SELECT assigned_by FROM Complaint WHERE assigned_by IS NOT NULL

)

ORDER BY s.role;

QUERY 12 : Find staff with salary above the average salary.

SELECT staff_id, name, role, salary

FROM Staff

WHERE salary > (SELECT AVG(salary) FROM Staff)

ORDER BY salary DESC;

QUERY 12 : List all open complaints with the student name and assigned staff name.

```
SELECT  c.comp_id,  c.type,  c.description,    c.reporting_date,  s.name    AS
student_name, st.name AS assigned_staff

FROM Complaint c

JOIN Student s ON c.student_id = s.student_id

LEFT JOIN Staff st ON c.assigned_by = st.staff_id

WHERE c.status = 'Open'

ORDER BY c.reporting_date;
```

QUERY 13 : Average time to resolve a complaint (from reporting_date to current if still open).

```
SELECT  comp_id, type, status, reporting_date,

CASE

    WHEN status = 'Solved' THEN 'Resolved'

    ELSE CONCAT(CURRENT_DATE - reporting_date, ' days pending')

END AS resolution_info

FROM Complaint
```

ORDER BY reporting_date;

QUERY 14 : Find all maintenance staff (Plumber/Carpenter/Electrician) and how many complaints each has resolved.

```
SELECT st.staff_id, st.name, st.role,
       COUNT(c.comp_id) AS complaints_resolved
FROM Staff st
LEFT JOIN Complaint c ON c.solved_by = st.staff_id AND c.status = 'Closed'
WHERE st.role IN ('Plumber', 'Carpenter', 'Electrician')
GROUP BY st.staff_id, st.name, st.role
ORDER BY complaints_resolved DESC;
```

QUERY 15 : List all complaints reported in a specific month (February 2025)

```
SELECT c.comp_id, c.type, c.status, c.description, s.name AS
student_name
FROM Complaint c
JOIN Student s ON c.student_id = s.student_id
WHERE EXTRACT(MONTH FROM c.reporting_date) = 2
      AND EXTRACT(YEAR FROM c.reporting_date) = 2025
```

ORDER BY c.reporting_date;

QUERY 16 : List all approved visitors with their allocated room and approving admin.

```
SELECT v.visitor_id, v.name AS visitor_name, v.contact_no, v.aadhar_no,  
v.check_in, v.check_out, v.stay_duration, r.room_no AS allocated_room,  
w.wing_name, a.name AS approved_by
```

```
FROM Visitor v
```

```
LEFT JOIN Rooms r ON v.allocated_room = r.room_no
```

```
LEFT JOIN Wing w ON r.wing_id = w.wing_id
```

```
LEFT JOIN Admin a ON v.approved_by = a.admin_id
```

```
WHERE v.request_status = 'Approved'
```

```
ORDER BY v.check_in;
```

QUERY 17 : List visitors currently checked in (check_in <= now AND check_out > now).

```
SELECT visitor_id, name, contact_no, allocated_room, check_in, check_out,  
stay_duration, FROM Visitor
```

```
WHERE check_in <= NOW()
```

AND check_out >= NOW()

AND request_status = 'Approved';

QUERY 18 : List all suggestions submitted by visitors, along with the visitor name.

SELECT sg.suggestion_id, v.name AS visitor_name, v.contact_no, sg.description

FROM Suggestions sg

JOIN Visitor v ON sg.visitor_id = v.visitor_id

ORDER BY sg.suggestion_id;

QUERY 18 : Count visitor requests handled by each admin.

SELECT a.admin_id, a.name AS admin_name,

COUNT(v.visitor_id) AS approved_visitors

FROM Admin a

LEFT JOIN Visitor v ON v.approved_by = a.admin_id

GROUP BY a.admin_id, a.name

ORDER BY approved_visitors DESC;

QUERY 19 : List visitors along with their payment details if they made a payment.

```
SELECT v.visitor_id, v.name AS visitor_name, vp.transaction_id, p.method AS  
payment_method, vp.amount, p.date AS payment_date  
  
FROM Visitor v  
  
JOIN Visitor_Payment vp ON vp.visitor_id = v.visitor_id  
  
JOIN Payment p ON p.transaction_id = vp.transaction_id  
  
ORDER BY p.date;
```

QUERY 20 : Total amount collected from students per month.

```
SELECT TO_CHAR(p.date, 'YYYY-MM') AS month,  
COUNT(sp.transaction_id) AS num_payments, SUM(sp.amount) AS total_collected  
  
FROM Student_Payment sp  
  
JOIN Payment p ON sp.transaction_id = p.transaction_id  
  
GROUP BY TO_CHAR(p.date, 'YYYY-MM')  
  
ORDER BY month;
```

QUERY 21 : Find students who have NOT made any payment.

```
SELECT s.student_id, s.name, s.branch, s.year, s.email  
  
FROM Student s
```

```
WHERE s.student_id NOT IN (  
  
    SELECT student_id FROM Student_Payment  
  
)  
  
ORDER BY s.name;
```

QUERY 22 : List parcels that are still pending (not yet handed to student) along with student contact details.

```
SELECT p.parcel_id, p.arrival_date, p.timestamp, p.status, s.name AS student_name,  
s.contact AS student_contact, r.room_no, w.wing_name  
  
FROM Parcel p  
  
JOIN Student s ON p.student_id = s.student_id  
  
JOIN Rooms r ON s.resident_of = r.room_no  
  
JOIN Wing w ON r.wing_id = w.wing_id  
  
WHERE p.status = 'Pending'  
  
ORDER BY p.arrival_date;
```

QUERY 23 : List all approved overnight leave requests with student name,dates, and approving warden.

```
SELECT ol.leave_id, s.name AS student_name, ol.from_date, ol.to_date,
ol.return_timestamp, ol.status, st.name AS approved_by_warden

FROM Overnight_Leave ol

JOIN Student s ON ol.student_id = s.student_id

JOIN Staff st ON ol.approved_by = st.staff_id

WHERE ol.status = 'Approved'

ORDER BY ol.from_date;
```

```
QUERY 23 : Find students with pending leave requests. SELECT
ol.leave_id, s.name AS student_name, s.contact AS student_contact,
ol.request_date, ol.from_date, ol.to_date, w.wing_name

FROM Overnight_Leave ol

JOIN Student s ON ol.student_id = s.student_id

JOIN Rooms r ON s.resident_of = r.room_no

JOIN Wing w ON r.wing_id = w.wing_id

WHERE ol.status = 'Pending'

ORDER BY ol.request_date;
```

QUERY 25 : List students who were absent on a specific date (2025-04-09).

SELECT s.student_id, s.name, s.contact, r.room_no, w.wing_name

FROM Attendance a

JOIN Student s ON a.student_id = s.student_id

JOIN Rooms r ON s.resident_of = r.room_no

JOIN Wing w ON r.wing_id = w.wing_id

WHERE a.date = '2025-04-09'

AND a.status = 'Absent'

ORDER BY w.wing_name;

QUERY 26 : Total payment collected from students vs visitors

SELECT 'Students' AS payer_type, SUM(amount) AS total_collected

FROM Student_Payment

UNION ALL

SELECT 'Visitors' AS payer_type, SUM(amount) AS total_collected

FROM Visitor_Payment;

QUERY 27 : Calculate attendance percentage for each student for the tracked week (2025-04-07 to 2025-04-11).

```
SELECT s.student_id, s.name, COUNT(a.date) AS total_days,  
       SUM(CASE WHEN a.status = 'Present' THEN 1 ELSE 0 END) AS days_present,  
       ROUND(  
           100.0 * SUM(CASE WHEN a.status = 'Present' THEN 1 ELSE 0 END) /  
           NULLIF(COUNT(a.date), 0), 2  
       ) AS attendance_pct  
FROM Student s  
JOIN Attendance a ON a.student_id = s.student_id  
WHERE a.date BETWEEN '2025-04-07' AND '2025-04-11'  
GROUP BY s.student_id, s.name  
ORDER BY attendance_pct;
```